Лабораторная работа №4
Динамическое программирование

Борисюк Е. Г. Вариант 4.

1. Условие и математическая постановка

Компания распределяет 7 одинаковых серверных модулей между тремя сервисами. Производительность сервисов складывается. Сервис обработки заказов должен получить не менее 1 модуля; ни один сервис не может получить более 3 модулей. Требуется распределить все 7 модулей так, чтобы получить максимальную суммарную производительность.

u	Обработка заказов	Аналитика	Поиск
0	0	0	0
1	6	2	4
2	10	7	9
3	13	12	13
4	16	16	15
5	18	19	17
6	19	21	18
7	20	23	19

Пусть x_1, x_2, x_3 - количество модулей у обработки заказов, аналитики и поиска соответственно; g_1, g_2, g_3 - табличные функции производительности.

$$Z = g_1(x_1) + g_2(x_2) + g_3(x_3) \to \max.$$

$$x_1 + x_2 + x_3 = 7;\ 1 \le x_1 \le 3;\ 0 \le x_2 \le 3;\ 0 \le x_3 \le 3;$$
$$x_1, x_2, x_3 \text{ - целые числа.}$$

Строки таблицы для u = 4, 5, 6, 7 сохраняются как часть условия, но не участвуют в допустимых управлениях: каждый сервис получает не более 3 модулей.

2. Этапы, состояния и уравнение Беллмана

Этап k соответствует сервису k, k = 1, 2, 3. Состояние s - число модулей, которое ещё требуется полностью распределить между сервисами k, …, 3. Управление u - число модулей, выделяемое текущему сервису. Следующее состояние равно s − u.

$F_k(s)$ - максимальная производительность сервисов k, …, 3 при точном распределении s модулей. Множества управлений: $A_1 = \{1, 2, 3\}$, $A_2 = A_3 = \{0, 1, 2, 3\}$.

$$F_k(s) = \max \{g_k(u) + F_{k+1}(s - u) : u \in A_k,\ u \le s\}.$$

Граница: $F_4(0) = 0$; $F_4(s) = -\infty$ при $s > 0$. Недостижимому состоянию соответствует $-\infty$. Максимум по пустому множеству равен $-\infty$. Кандидаты с недостижимым продолжением исключаются.

3. Обратный проход: поиск и аналитика

Условная оптимизация выполняется от последнего сервиса к первому: сначала F_3, затем F_2 и F_1. В $U_k(s)$ сохраняются все управления, достигающие максимума.

Этап 3. Остаток s полностью получает поиск, поэтому $F_3(s) = g_3(s)$ при $0 \le s \le 3$; при $s \ge 4$ состояние недостижимо.

s	0	1	2	3	4	5	6	7
$F_3(s)$	0	4	9	13	$-\infty$	$-\infty$	$-\infty$	$-\infty$
$U_3(s)$	{0}	{1}	{2}	{3}	∅	∅	∅	∅

Этап 2. Для каждого остатка перебираем допустимое число модулей аналитики. В столбце кандидатов перечислены $g_2(u) + F_3(s - u)$ в порядке возрастания допустимого u; недостижимые продолжения не включены.

s	Допустимые u	Значения кандидатов	$F_2(s)$	$U_2(s)$
0	{0}	0	0	{0}
1	{0, 1}	4; 2	4	{0}
2	{0, 1, 2}	9; 6; 7	9	{0}
3	{0, 1, 2, 3}	13; 11; 11; 12	13	{0}
4	{1, 2, 3}	15; 16; 16	16	{2, 3}
5	{2, 3}	20; 21	21	{3}
6	{3}	25	25	{3}
7	∅	нет	$-\infty$	∅

Например, при s = 4 допустимы u = 1, 2, 3:

$$F_2(4) = \max\{2 + 13;\ 7 + 9;\ 12 + 4\} = \max\{15;\ 16;\ 16\} = 16.$$

Здесь сохраняются два управления: $U_2(4) = \{2, 3\}$. При s = 5 получаем $\max\{7 + 13;\ 12 + 9\} = 21$, поэтому $U_2(5) = \{3\}$. При s = 6 допустимо только u = 3: $F_2(6) = 12 + 13 = 25$. Распределить 7 модулей между двумя сервисами с верхней границей 3 невозможно.

4. Обратный проход: обработка заказов

На первом этапе $u \in \{1, 2, 3\}$. Используем уже найденные $F_2(s - u)$. Для $s = 0$ нет допустимого управления; $F_1(0) = -\infty$, $U_1(0) = \emptyset$.

s	Допустимые u	Значения кандидатов	$F_1(s)$	$U_1(s)$
1	{1}	6	6	{1}
2	{1, 2}	10; 10	10	{1, 2}
3	{1, 2, 3}	15; 14; 13	15	{1}
4	{1, 2, 3}	19; 19; 17	19	{1, 2}
5	{1, 2, 3}	22; 23; 22	23	{2}
6	{1, 2, 3}	27; 26; 26	27	{1}
7	{1, 2, 3}	31; 31; 29	31	{1, 2}

В частности, для исходного состояния $s = 7$:

$$F_1(7) = \max\{6 + F_2(6); 10 + F_2(5); 13 + F_2(4)\}$$
$$= \max\{6 + 25; 10 + 21; 13 + 16\} = 31.$$

Следовательно, $U_1(7) = \{1, 2\}$. Обе ветви необходимо сохранить.

5. Прямой проход: восстановление стратегий

Начинаем с $k = 1$, $s = 7$ и идём по сохранённым управлениям, каждый раз заменяя s на $s - u$.

Ветвь	Этап 1	Этап 2	Этап 3	План
1	s=7, u=1	s=6, u=3	s=3, u=3	(1, 3, 3)
2	s=7, u=2	s=5, u=3	s=2, u=2	(2, 3, 2)

В обеих ветвях остаток после третьего этапа равен нулю. Управления на этапах 2 и 3 в этих состояниях единственны, поэтому других оптимальных стратегий нет. Равенство в $U_2(4)$ относится к неоптимальной для исходной задачи ветви $x_1 = 3$.

6. Ответ и проверка

Максимальная суммарная производительность равна 31. Оптимальны два распределения: (1, 3, 3) и (2, 3, 2).

$$g_1(1) + g_2(3) + g_3(3) = 6 + 12 + 13 = 31;$$
$$g_1(2) + g_2(3) + g_3(2) = 10 + 12 + 9 = 31.$$

Суммы модулей: $1 + 3 + 3 = 7$ и $2 + 3 + 2 = 7$. Обработка заказов получает хотя бы 1 модуль, все компоненты целые и не превышают 3.

7. Псевдокод основного алгоритма

Обозначения: n - число сервисов, R - общий ресурс, A[k] - допустимые управления. Нумерация этапов здесь начинается с 1.

```
Для всех k = 1, …, n+1 и s = 0, …, R: F[k,s] ← −∞
F[n+1,0] ← 0; все списки U[k,s] ← пустые
Для k от n до 1:
  Для s от 0 до R:
    Для каждого u из A[k], где u ≤ s:
      Если F[k+1,s−u] = −∞: пропустить u
      v ← g[k,u] + F[k+1,s−u]
      Если v > F[k,s]: F[k,s] ← v; U[k,s] ← [u]
      Иначе если v = F[k,s]: добавить u в U[k,s]
Восстановить(k,s):
  Если k = n+1: вернуть [пустой план] при s=0, иначе []
  plans ← пустой список
  Для каждого u из U[k,s]:
    Для каждого tail из Восстановить(k+1,s−u):
      добавить (u, tail) в plans
  Вернуть plans
Ответ: F[1,R] и Восстановить(1,R)
```

8. Программная реализация и диаграмма классов

Алгоритм реализован самостоятельно на Python без дополнительных библиотек. Один класс ResourceAllocation хранит исходные данные, значения подзадач и все оптимальные управления. В коде этапам 1, 2, 3 соответствуют индексы 0, 1, 2.

ResourceAllocation
total: int; profits: list[list[int]]; lower, upper: list[int]
n: int; values: list[list[number]]; choices: list[list[list[int]]]
__init__(total, profits, lower, upper)
solve() → number
restore(k=0, s=None) → list[tuple]

solve() выполняет обратный проход и возвращает максимум; restore() выполняет прямое восстановление всех планов. Внешние функции show_result(problem) и main() отвечают за вывод и запуск; они не являются методами класса. Поля values и choices заполняются в solve().

9. Запуск, результат и ссылки

Локальный запуск из каталога lab-04: python python/dynamic_programming.py. Ноутбук Colab содержит тот же класс, отдельный блок исходных данных и сохранённый вывод. Для запуска достаточно выбрать «Среда выполнения → Выполнить все».

Таблица / s	0	1	2	3	4	5	6	7
F3	0	4	9	13	−∞	−∞	−∞	−∞
F2	0	4	9	13	16	21	25	−∞
F1	−∞	6	10	15	19	23	27	31

Фактический результат запуска: максимум 31; стратегии (1, 3, 3) и (2, 3, 2). Вывод сохранён в python/result.txt. Независимый перебор шести допустимых распределений подтвердил значение максимума и полный список оптимальных планов.

Проверка перебором (используется только для контроля результата):

Распределение	Производительность
(1, 3, 3)	31
(2, 2, 3)	30
(2, 3, 2)	31
(3, 1, 3)	28
(3, 2, 2)	29
(3, 3, 1)	29

Репозиторий: lab-04, код и результат запуска

Открыть выполненный ноутбук в Google Colab

Исходное задание: Тема №4, задача 4

10. Рефлективный вывод

В работе удалось связать задачу распределения ресурса с состояниями и переходами динамического программирования. Основная трудность - учитывать точное распределение всего ресурса и недостижимые остатки. Значение −∞ позволяет исключить такие продолжения из максимизации. Ещё один важный момент - сохранять все равные максимумы: иначе можно потерять вторую оптимальную стратегию. Таблицы ручного решения и программный расчёт совпали; перебор допустимых планов подтвердил полноту ответа.